BUDGETPRO

PERFORMANCE / SECURITY / ENGINEERING

PROJECT REVIEW

Performance and Security Review of Using AI

React frontend / Node.js + Express / PostgreSQL

PROJECT

BudgetPro

PREPARED BY

Sreejith P R

DATE

20 October 2026

Process Overview

Project: BudgetPro | Scope: React frontend, Node.js/Express API, PostgreSQL

REVIEW TYPE

AI-assisted, source-based assessment of the checked-in implementation. This is not a penetration test, dependency audit, or measured load test.

01 / PROCESS OVERVIEW

The review traced browser input through API validation and database operations, then back to React state and rendering. Separate, bounded prompts were used for each layer so findings could be tied to observable code rather than assumptions.

REACT FRONTEND PROMPT

"Trace authentication and transaction data flow in the React client. Check browser storage, API error handling, dashboard calculations, and whether rendering or data transfer grows with account history. Cite implementation evidence and separate current behavior from scale risk."

EXPRESS / SECURITY PROMPT

"Review routes, session cookies, authentication, validation, SQL parameterization, authorization boundaries, error handling, and HTTP protections. For each concern, state preconditions, impact, confidence, and mitigation; do not claim exploitability without evidence."

POSTGRESQL / SCALE PROMPT

"Inspect schema, constraints, indexes, startup behavior, and query shapes. Estimate how transaction volume and concurrency affect query work, response size, connection demand, and financial precision. Recommend incremental changes supported by observed design."

CROSS-LAYER SYNTHESIS PROMPT

"Combine findings without duplication. Rank security concerns by severity and confidence; classify each as observed control, observed gap, or conditional risk. Rank performance actions by expected bottleneck and identify what requires measurement."

HOW OUTPUTS WERE ANALYZED

1 / Establish evidence

Read the source files and map claims to routes, components, queries, or schema constraints. Avoid unsupported assumptions.

2 / Trace end to end

Follow input -> validation -> identity/authorization -> SQL -> response -> React render. Confirm ownership at the API boundary.

3 / Assess risk

Consider preconditions and impact; rank severity and confidence. Distinguish direct defects from defense-in-depth gaps.

4 / Assess scale

Compare query work and returned rows with account growth; inspect indexing and connection behavior. Treat forecasts as hypotheses.

Prompts document the review questions used to structure this report; they are not claimed as a verbatim transcript of earlier prompt history. Validate findings against deployment configuration and runtime tests.

Security analysis

No critical or high-severity exploitable defect was confirmed in reviewed source. Findings below are code-level hardening gaps or conditional risks to address before handling real financial data at scale.

MEDIUM / HARDENING - No visible auth rate limit

Registration and login accept requests without throttling, lockout, or abuse controls. This can enable high-volume password guessing, credential stuffing, and signup abuse; no successful compromise was demonstrated. Evidence: server/index.js:132-169. Mitigate with per-IP and per-account throttling, progressive backoff, and monitoring.

MEDIUM / CONDITIONAL - No explicit CSRF/origin check

Transaction creation relies on an automatically sent session cookie. SameSite=Lax and allowlisted CORS help, but CORS is not a general CSRF defense; same-site cross-origin contexts (such as a compromised sibling subdomain) may still matter. Evidence: server/index.js:15-20, 49, 202-222. Verify request Origin/Fetch Metadata and add a CSRF token if deployment requires cross-site cookies.

LOW / DEFENSE-IN-DEPTH - No visible security headers

Production serves static assets without visible Helmet/security-header middleware. CSP, HSTS, clickjacking, and MIME-sniffing policies are not set in this app code; an upstream proxy may already supply some. Evidence: server/index.js:226-229. Configure Helmet/CSP and HSTS at the HTTPS edge; verify deployed responses.

LOW / CONDITIONAL - Seven-day JWT cannot be revoked

Logout clears the browser cookie, but a copied JWT remains valid until expiry. HttpOnly reduces script access; no token theft was observed. Evidence: server/index.js:60-61, 166-169. Shorten access-token life and add revocation/rotation if prompt invalidation is required.

POSITIVE CONTROLS OBSERVED

IN PLACE	Passwords hashed with bcrypt (cost 12); raw passwords are not persisted.
IN PLACE	JWT held in HttpOnly, SameSite=Lax cookie; Secure enabled in production mode.
IN PLACE	Parameterized transaction SQL is scoped to the authenticated user ID.
IN PLACE	Request payloads are validated; database constraints reinforce invariants.

Scope limit: static source review only. No live database, production headers/proxy, browser CSRF test, penetration test, or independent dependency scan was assessed.

Performance considerations

The schema includes a useful per-user/date index. The largest expected scaling pressure is unbounded transaction-history retrieval - not rendering the eight recent rows.

1 / Full history is fetched and summarized in React

GET /api/transactions returns every user row; React downloads the full list, then filters the month and calculates totals. Only eight rows are displayed, but all history is transferred and retained. Response bytes, query work, memory, and dashboard latency grow with account history. Evidence: server/index.js:186-198; src/App.jsx:190-210, 235-250, 280-299.

2 / Aggregates run after full transfer

Monthly totals and expense-category totals are computed in JavaScript after all transactions arrive. Keep aggregate work in SQL and separate it from transaction history. Add date-bounded summary queries and a size-capped recent-transaction endpoint with deterministic cursor pagination; align indexes to the filters and sort.

3 / Connection demand scales per API process

The pg Pool uses its default per-process limit. Replicas multiply possible connections and can exhaust PostgreSQL capacity before application CPU is saturated. Set max/acquisition/idle limits against the DB budget; monitor wait time and active connections; consider PgBouncer when needed.

4 / Schema DDL is coupled to API startup

Each API process executes CREATE TABLE / index DDL before serving. This can race across replicas and requires schema privileges on the runtime account. Move changes to versioned, single-run migrations and restrict the runtime DB role to needed data permissions.

5 / Amounts cross into binary floating point

PostgreSQL stores NUMERIC(12,2), but the API casts to float8 and React sums floating-point values. Rounding drift can appear in larger aggregates. Keep amounts/aggregates as NUMERIC decimal strings or use integer minor units consistently; test rounding behavior.

RECOMMENDED ROLLOUT

NOW:	Measure response size and p50/p95 latency with representative small and large histories; add query and pool metrics.
BEFORE GROWTH:	Split SQL summaries from paginated history; cap page size and use stable cursors. Verify plans with EXPLAIN (ANALYZE, BUFFERS) on realistic data.
AT HIGHER CONCURRENCY:	Load-test realistic read/write mixes, tune pool limits to PostgreSQL capacity, and add pooling/caching only when measurements justify it. Re-test user isolation and totals.

Conclusion: appropriate for a small application. Prioritize bounded data access and SQL summaries before transaction history grows; use production-like measurements to decide when additional pooling, caching, or indexing is warranted.